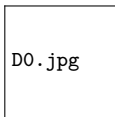


Lecture 14: Domain Relational Calculus & QBE

Domain Variables and Visual Querying

Z2004: Database Management Systems



IIT Madras Zanzibar

Monday, 30 March 2026

1. TRC vs. DRC: The Pizza Analogy

Tuple Calculus (TRC)

In TRC, our variable t represents an **entire row** (a tuple).

The Analogy: It's like grabbing a whole pizza, and then specifying you only want to look at the cheese: $t.Cheese$.

- ▶ $\{ t.Name \mid t \in Student \}$
- ▶ The query must fetch the whole row first to check attributes.

Domain Calculus (DRC)

In DRC, variables represent **individual column values** (domains).

The Analogy: It's like going straight to the ingredient bins. You don't grab a pizza; you grab a handful of *cheese* and *crust*.

- ▶ $\{ \langle n \rangle \mid Student(s, n, d, g) \}$
- ▶ We define a variable for every single column right away.

2. DRC Syntax & Our Schema

The General Form

$$\{ \langle x_1, x_2, \dots, x_k \rangle \mid P(x_1, x_2, \dots, x_k) \}$$

- ▶ $x_1, \dots, x_k$: Domain variables representing single column values.
- ▶ $\langle \dots \rangle$: Angle brackets denote the **list of variables** we want to output (Project).

Our University Schema

Student(SID, Name, Dept, GPA)

Enrolment(SID, CID, Grade)

Course(CID, CName, Credits)

s n d g

s c gr

c cn cr

i Notice how we use the **same variable 's'** for *SID* in both *Student* and *Enrolment*!

2b. Deconstructing the Symbols

Breaking Down the Formula

Let's look closely at a basic DRC query and map it to our schema:

$$\{ \langle \mathbf{n} \rangle \mid \text{Student}(s, \mathbf{n}, d, g) \}$$

 **{ }** (**Set Brackets**): Indicates that the query will return a *set* of results.

 **$\langle n \rangle$** (**Target List**): The angle brackets hold the variable(s) we want to **output** (Project). Because we defined n as the second position in the Student table, n represents the **Name**.

Think of this as the SELECT clause in SQL: SELECT Name

 **|** (**Such That**): The vertical bar separates our desired output from our logical conditions. *Think of this as the WHERE clause.*

 **Student(s, n, d, g)** (**The Atom**): This asserts that the combination of variables (s, n, d, g) must exactly match a valid row in the **Student** table.

3. DRC Examples: Filtering & Projection

Example 1: Filtering

Goal: Get all info for students with a GPA > 3.5 .

$$\{\langle s, n, d, g \rangle \mid \text{Student}(s, n, d, g) \wedge g > 3.5\}$$

How to read it:

"Output the values $\langle s, n, d, g \rangle$, such that these values form a valid row in the Student table, AND the g value is greater than 3.5."

Example 2: Unused Variables

Goal: Get ONLY the **Names** of 'CS' students.

$$\{\langle n \rangle \mid \exists s \exists d \exists g (\text{Student}(s, n, d, g) \wedge d = \text{'CS'})\}$$

⚠ The $\exists$ Rule:

Because we only output $\langle n \rangle$, the other variables (s, d, g) are left hanging. Any variable inside the formula but NOT in the output brackets **must** be bound by an Existential Quantifier ($\exists$).

3. DRC Examples: The Implicit Join

Example 3: Cross-Table Joins

Goal: Find the **Names** of all students enrolled in course 'C101'.

$$\{ \langle n \rangle \mid \exists s \exists d \exists g \exists gr (Student(s, n, d, g) \wedge Enrolment(s, 'C101', gr)) \}$$

The Magic of Shared Variables

- ▶ Notice we used the variable *s* in BOTH the Student atom and the Enrolment atom.
- ▶ In DRC, if you use the exact same variable name in two different tables, the math **forces them to be equal**.
- ▶ This is an implicit JOIN!
(*Student.SID* = *Enrolment.SID*).

SQL Equivalent

```
SELECT DISTINCT s.Name  
FROM Student s  
JOIN Enrolment e ON s.SID = e.SID  
WHERE e.CID = 'C101';
```

3b. Side-by-Side: TRC vs. DRC

Comparing the Paradigms

Goal: Find the **Names** of all students who have a GPA > 3.5 .

1. Tuple Relational Calculus

Variables represent **entire rows**.

$$\{ t.Name \mid t \in Student \wedge t.GPA > 3.5 \}$$

3. Standard SQL

SQL is basically TRC in plain English!

```
SELECT Name
FROM Student
WHERE GPA > 3.5;
```

2. Domain Relational Calculus

Variables represent **single columns**.

$$\{ \langle n \rangle \mid \exists s \exists d \exists g (Student(s, n, d, g) \wedge g > 3.5) \}$$

Why use DRC?

Although TRC maps well to SQL, DRC maps perfectly to visual query interfaces, as we will see in Query By Example (QBE)!

4. QBE — Query By Example

What is QBE?

- ▶ Invented by Moshe Zloof at IBM Research in 1970.
- ▶ It is a **visual, form-based** query language directly built on top of Domain Relational Calculus.
- ▶ Instead of writing code, the user is presented with a visual grid (a skeleton table) and types variables/conditions into the columns.
- ▶ This is the grandfather of the visual query builders in MS Access and Tableau!

Notation Rules

- ▶ **P.** → "Print" (Output this column).
- ▶ **_var** → Defines a variable (Starts with underscore).
- ▶ **> 3.5** → A condition/filter.

Visualizing QBE

Query: "Names of CS students with GPA > 3.5"

SID	Name	Dept	GPA
	P. _n	CS	>3.5

Breakdown:

- ▶ The system leaves SID blank (we don't care about it).
- ▶ It Prints (**P.**) the Name variable (**_n**).
- ▶ It filters Dept to EXACTLY **CS**.
- ▶ It filters GPA to **>3.5**.

4b. QBE: Performing a Join

Linking Tables Visually

Just like in DRC, QBE performs an implicit JOIN when you use the **exact same variable name** across two different tables.

Goal: Print the **Names** of students enrolled in course '**C101**'.

SID	Name	Dept	GPA
_x	P._n		

SID	CID	Grade
_x	C101	

 The variable `_x` is placed in the SID column of *both* tables.

 The database engine sees this and automatically performs: `Student.SID = Enrolment.SID`.

4c. QBE: Negation (NOT)

The NOT Operator ($\neg$)

To specify that a record must **NOT** exist, QBE uses a $\neg$ (or NOT) placed directly under the table name.

Goal: Print the names of students who are **NOT** enrolled in any course.

SID	Name	...
$_x$	P. $_n$	

$\neg$ Enrolment	SID	...
	$_x$	

How it works

- 1 We grab a student's ID and assign it to $_x$.
- 2 We check the Enrolment table for $_x$.
- 3 The $\neg$ symbol tells the system: *"Only keep this student if you CANNOT find $_x$ in this table."*

💡 DRC Equivalent:

$$\{\langle n \rangle \mid \exists s \exists d \exists g (\text{Student}(s, n, d, g) \wedge \neg(\exists c \exists gr \text{ Enrolment}(s, c, gr)))\}$$

4d. QBE: The "OR" Condition

Using Multiple Rows

We already saw that putting conditions in the **same row** acts as an **AND** operator. But what if we want an **OR** condition? In QBE, we simply add another row to our grid!

Goal: Print the names of students who are in the 'CS' department **OR** have a **GPA > 3.8**.

SID	Name	Dept	GPA
	P. n	CS	
	P. n		>3.8

How it works

- ✓ The database engine evaluates each row independently.
- ✓ If a student matches **Row 1** (they are in CS), their name is printed.
- ✓ If a student matches **Row 2** (their GPA is > 3.8), their name is printed.

SQL Equivalent

```
SELECT Name
FROM Student
WHERE Dept = 'CS'
      OR GPA > 3.8;
```

Class Activity: DRC and QBE

The Data Schema

We are using our university tables:

- ▶ **Student**(SID, Name, Dept, GPA)
- ▶ **Enrolment**(SID, CID, Grade)



Instructions

Take 5 minutes to solve the two tasks on the right.
Discuss with your neighbor if needed!

Task 1: Read the DRC

Translate this mathematical expression into plain English. What is it asking the database to do?

$$\{\langle n \rangle \mid \exists s \exists d \exists g (\text{Student}(s, n, d, g) \wedge g = 4.0)\}$$

Task 2: Write the QBE

Draw a simple QBE grid for the following request:

"Print the Grade of the student whose SID is 'S10' in the course 'C50'."

Class Activity: Solutions

Task 1 Solution

The Query:

$\{\langle n \rangle \mid \exists s \exists d \exists g (\text{Student}(s, n, d, g) \wedge g = 4.0)\}$

Plain English Translation:

*"Retrieve the **Names** ($\langle n \rangle$) of all students who have a perfect **GPA** of 4.0."*

SQL Equivalent:

```
SELECT Name
FROM Student
WHERE GPA = 4.0;
```

Task 2 Solution

The Prompt:

"Print the Grade of the student whose SID is 'S10' in the course 'C50'."

The QBE Grid:

We only need the **Enrolment** table for this!

Enrolment		
SID	CID	Grade
S10	C50	P. _gr

Breakdown:

We put exact constants ('S10' and 'C50') under their columns, and put the Print command (**P.**) alongside a variable (**_gr**) under Grade.

Key Takeaways

- ✓ **DRC Variables:** Variables represent single *domains* (column values), not entire rows.
- ✓ **The Format:** $\{\langle n \rangle \mid \text{Student}(s, n, d, g)\}$.
- ✓ **Implicit Joins:** Sharing the exact same variable across two tables automatically joins them.
- ✓ **Unused Variables:** Must be bound with $\exists$.
- ✓ **QBE (Query By Example):** IBM's visual GUI based entirely on DRC. It uses skeleton tables and the **P**. notation to project data visually.

Next Lecture (L15)

Database Normalization

- ➔ Moving from Theory to Design!
- ➔ Data Redundancy & Anomalies.
- ➔ Functional Dependencies.
- ➔ First, Second, and Third Normal Forms (1NF, 2NF, 3NF).

Reading Assignment

- 📖 Read **Silberschatz, Ch. 8** (Relational Database Design) to prepare.